Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema – “Python web”

A efectuat : st. gr.SI – 264 Danuța Daniel

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

1.Part 1 – Imports

```python
# Laborator: funcții, metode și importuri pe web
# Student: Danuta Daniel

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde


# --Ex1--

import requests
#print(requests.__version__)

import urllib.request

# libraria "request" este una third party
# pe cand libraria urllib este una ce vine deja inclusa
# in interpretatorul python si este o librarie standard

# Output: 2.32.5


# --Ex2--

#print(requests.get(BASE_URL))

#<Response [200]>

#from requests import get
#response = get(BASE_URL)
#print(response)

#<Response [200]>

# Stilul unu este usor de inteles deoarece se vede imediat de unde
# vine functia "requests.get" si nu poate fi confundat

# Stilul doi are avantajul ca apelam direct la get
# fara prefixul request , folositor cand folosim functia
# de mai multe ori


# --Ex3--

#import requests as rq
#print(rq.get(BASE_URL))

# Un alias face codul mai usor de citit cand numele librariei
# este unul lung sau cand evita confuzia intre mai multe functii

# Un alias face codul mai greu cand acelasi alias inseamna alte
# lucruri sau cand numele original este deja scurt si aliasul nu ajuta


# --Ex4--

#with urllib.request.urlopen(ECHO_URL) as response:
    #print(response.status)
    #body = response.read(200).decode('utf-8')
    #print(body)

# Output: 200
# <!DOCTYPE html>
# <html lang="en">
#
# <head>
#     <meta charset="UTF-8">
#     <title>httpbin.org</title>
#     <link href="https://fonts.googleapis.com/css?family=Open+Sans:400,700|Source+Code+Pro:300,600|Tit


```

Fig 1 – imports/1

```
71
72  # --Ex5--
73
74  #print(dir(requests))
75
76  # Class: Session, folosita ca requests.Session(),
77  # pastreaza cookieurile si setarile dintro sesiune
78
79  # Function: get, se apeleaza cu requests.get()
80
81  # Module: adapters, un fisier .py din pachetul requests care
82  # contine clase si functii
83
84
85  # --Ex6--
86
87  #help(requests.get)
88  #requests.get(BASE_URL, timeout=5)
89  # Parametrul care seteaza timpul maxim de asteptare
90  # este timeout care este folosit in modelul de mai sus .get(url, param)
91
92
93  # --Ex7--
94
95  # import time
96  # start = time.perf_counter()
97  # response = requests.get(BASE_URL, timeout=TIMEOUT)
98  # end = time.perf_counter()
99  #
100 # print(end - start)
101 # print(response.elapsed.total_seconds())
102
103
104 #0.830484208 - time.perf_counter()
105 #0.810986 - elapsed.total_seconds()
106
107
108 # --Ex8--
109
110 # try:
111 #     import bs4
112 #
113 # except ImportError:
114 #     print("Instalați modulul cu: pip install beautifulsoup4")
115
116
117 # ------------
```

Fig 2 – imports/2

2.Part 2 – Methods

```python
# Laborator: funcții, metode și importuri pe web
# Student: Danuta Daniel

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

import requests

# --Ex9--

# response = requests.get(BASE_URL, timeout=TIMEOUT)
#
# print(response.status_code)  # atribut
# print(response.ok)           # atribut
# print(response.url)          # atribut
# print(response.encoding)     # atribut
#
# 200
# True
# https://cybercor.org/
# utf-8


# --Ex10--

import requests

response = requests.get(BASE_URL, timeout=TIMEOUT)
# response.raise_for_status()
#
# try:
#     response = requests.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
#     response.raise_for_status()
# except requests.HTTPError as e:
#     print("Aceasta pagina nu a fost gasita sau serverul a raspuns cu o eroare.")
#     print("Detalii:", e)


# --Ex11--

# for name, value in response.headers.items():
#     print(f"{name}: {value}")

# --Ex12--

# print(response.headers.get("Server", "lipsește"))
# print(response.headers.get("Content-Type", "lipsește"))
#
# print(response.headers.get("content-type", "lipsește"))

# Observam ca response.headers este un dictionar care nu este
# case sensitive si poate fi folosit cu atat litere mari cat si cu
# litere mici


# --Ex13--

# print(response.text.lower().count("cyber"))

# Putem inlantuii aceste comenzi deoarece fara .lower
# cuvantul cyber cu alte capitalizari nu ar fi contate la .count()

# --Ex14--

# html = response.text
#
# start = html.find("<title>") + len("<title>")
# end = html.find("</title>")
#
# title = html[start:end].strip()
# print(title)
```

Fig 3 – Methods/1

```
77
78      # --Ex15--
79
80      # lines = response.text.splitlines()
81      #
82      # print("Nr de linii:", len(lines))
83      #
84      # longest = max(lines, key=len)
85      # print("Lungimea celei mai lungi linii:", len(longest))
86
87
88      # --Ex16--
89
90      # if response.url.startswith("https://"):
91      #     print("Conexiune securizată")
92      # else:
93      #     print("Conexiune nesecurizată")
94
95      # Output: Conexiune securizată
96
97
98      # --Ex17--
99
100     # response2 = requests.get("http://cybercor.org", timeout=TIMEOUT)
101     #
102     # for redirect in response2.history:
103     #     print(redirect.status_code, redirect.url)
104     #
105     # print("URL fin:", response2.url)
106
107     # Output: 301 http://cybercor.org/
108     # URL fin: https://cybercor.org/
109
110
111     # --Ex18--
112
113     # head_response = requests.head(BASE_URL, timeout=TIMEOUT)
114     # get_response = requests.get(BASE_URL, timeout=TIMEOUT)
115     #
116     # print("HEAD:", len(head_response.content))
117     # print("GET: ", len(get_response.content))
118
119     # HEAD cere doar informatia despre pagina , nu si pagina
120     # GET aduce si corpul astfel arata dimensiunea reala
121
122
123     # --Ex19--
124
125     # if response.cookies:
126     #     for cookie in response.cookies:
127     #         print(cookie.name, cookie.secure)
128     # else:
129     #     print("Niciun cookie setat")
130
131
132     # --Ex20--
133
134     # session = requests.Session()
135     # session.headers.update({"User-Agent": "WebLab-Daniel"})
136     #
137     # response = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
138     # print(response.json())
139     # print(response.text)
140
141     #--------------------------------
```

Fig 4 – Methods/2

3.Part 3 – Functions

```python
# Laborator: funcții, metode și importuri pe web
# Student: Danuta Daniel
BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde
import requests
import time
# --Ex21--

# def fetch(url):
#     return requests.get(url, timeout=TIMEOUT)
#
# response = fetch(BASE_URL)
# print(response.status_code)


# --Ex22--

def get_status(url: str) -> int:
    return requests.get(url, timeout=TIMEOUT).status_code
#
# for path in ["/", "/robots.txt", "/sitemap.xml"]:
#     print(path, get_status(BASE_URL + path))
#     time.sleep(1)

# Output: / 200
# /robots.txt 200
# /sitemap.xml 200


# --Ex23--

def fetch(url, timeout=10) -> requests.Response:
    return requests.get(url, timeout=timeout)


# response1 = fetch(BASE_URL)
# print(response1.status_code)
#
# response2 = fetch(BASE_URL, timeout=3)
# print(response2.status_code)


# --Ex24--

# def get_title(html):
#     start = html.find("<title>") + len("<title>")
#     end = html.find("</title>")
#     return html[start:end].strip
#
#
# response = fetch(BASE_URL)
# print(get_title(response.text))


# --Ex25--

def get_title(html:str) -> str:
    """Extrage titlul paginii dintr-un sir html
    Primeste codul html ca sir de caractere si extrage
    doar textul intre <title> si </title> , iar .striped
    elimina spatiile in cazul in care sunt prezente.
    """
    start = html.find("<title>") + len("<title>")
    end = html.find("</title>")
    return html[start:end].strip

# help(get_title)

```

Fig 5 – Functions/1

```python
# --Ex26--

# print(get_status(123))

# Adnotarile nu ne opresc , adnotarile sunt dor indice
# python nu le verifica la rulare


# --Ex27--

def page_exists(url: str) -> bool:
    try:
        return get_status(url) == 200
    except requests.RequestException:
        return False

# print(page_exists(BASE_URL))
# print(page_exists("https://this-domain-does-not-exist.invalid"))

# Output: True
# False


# --Ex28--

def check_paths(base, paths):
    results = {}
    for path in paths:
        results[path] = get_status(base + path)
        time.sleep(1)
    return results


# statuses = check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"])
# print(statuses)


# --Ex29--

def get_header(url, name, default = "lipsește"):
    response = requests.get(url, timeout=TIMEOUT)
    return response.headers.get(name, default)


# print(get_header(BASE_URL, name="Server"))


# --Ex30--

SECURITY_HEADERS = [
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "X-Frame-Options",
    "X-Content-Type-Options",
    "Referrer-Policy",
]


def security_headers(url: str) -> dict:
    response = requests.get(url, timeout=TIMEOUT)
    return {name: name in response.headers for name in SECURITY_HEADERS}

#result = security_headers(BASE_URL)
# print(result)
```

Fig 6 – Functions/2

```python

# --Ex31--

def score_headers(results: dict) -> str:
    present = sum(results.values())
    total = len(results)
    return f"{present}/{total}"

# print(score_headers(security_headers(BASE_URL)))

# Output = 3/5


# --Ex32--

def fetch_robots(base):
    response = requests.get(base + "/robots.txt", timeout=TIMEOUT)
    if response.status_code != 200:
        return None
    return response.text

def disallowed_paths(robots_text):
    if robots_text is None:
        return []
    paths = []
    for line in robots_text.splitlines():
        if line.startswith("Disallow:"):
            paths.append(line.split(":")[1].strip())
    return paths

# robots = fetch_robots(BASE_URL)
# print(disallowed_paths(robots))
# print(robots)


# --Ex33--

def response_times(*urls):
    times = {}
    for url in urls:
        response = requests.get(url, timeout=TIMEOUT)
        times[url] = response.elapsed.total_seconds()
    return times

# print(response_times(BASE_URL, ECHO_URL))

# Output: {'https://cybercor.org': 0.287727, 'https://httpbin.org': 0.534556}


# --Ex34--

def log(message, **details):
    parts = [message]
    for key, value in details.items():
        parts.append(f"{key}={value}")
    print(" | ".join(parts))

# log("verificat", url=BASE_URL, status=200)

# print() arata valoarea pe ecran pentru utilizator , dupa afisare
# valoarea nu ajunge la codul care a apelat functia

# return trimite valoarea inapoi catre codul care a apelat functia
# astfel poate fi salvata intro variabila , folosita de alte functii
```

Fig 7 – Functions/3

4.Part 4 – Modules

```python
# Laborator: funcții, metode și importuri pe web
# Student: Danuta Daniel
from part3_functions import fetch

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

import socket
from urllib.parse import urlparse , urljoin
import requests
import time
import json
import ssl

# --Ex35--

url = "https://cybercor.org/path?x=1#top"
parts = urlparse(url)

# print(parts.scheme)
# print(parts.netloc)
# print(parts.path)
# print(parts.query)
# print(parts.fragment)

# Output:
# https
# cybercor.org
# /path
# x=1
# top


# --Ex36--

# from urllib.parse import urljoin
#
# for link in ["/about", "contact.html", "../index.html"]:
#     print(urljoin(BASE_URL, link))


# --Ex37--

import re

def extract_links(html):
    links = re.findall(r'href="([^"]+)"', html)
    unique = []
    for link in links:
        if link not in unique:
            unique.append(link)
    return unique

# response = fetch(BASE_URL)
# print(extract_links(response.text))

# Output: ['/assets/index-CErP0EKt.css']

```

Fig 8 – Modules/1

```python

# --Ex38--

def split_links(links: list, domain: str) -> tuple:
    """Întoarce (interne, externe): legăturile de pe același domeniu și cele către alte site-uri."""
    internal = []
    external = []
    for link in links:
        full = urljoin(BASE_URL, link)       # 1. legătura devine URL complet
        netloc = urlparse(full).netloc       # 2. extragem domeniul
        if netloc == domain:
            internal.append(full)
        else:
            external.append(full)
    return internal, external


# links = extract_links(fetch(BASE_URL).text)
# internal, external = split_links(links, "cybercor.org")
# print("Interne:", internal)
# print("Externe:", external)


# --Ex39--

from html.parser import HTMLParser


class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)

# test_html = """
# <html><body>
#   <img src="/logo.png" alt="Logo">
#   <IMG SRC="poza.jpg">
#   <img alt="imagine fără src">
#   <img src="https://justcats.ie/wp-content/uploads/2024/10/Untitled-design-1.webp" />
#   <a href="/despre">Aceasta nu este o imagine</a>
# </body></html>
# """
#
# finder = ImageFinder()
# finder.feed(test_html)
# print(finder.images)
#
# assert finder.images == [
#     "/logo.png",
#     "poza.jpg",
#     "https://justcats.ie/wp-content/uploads/2024/10/Untitled-design-1.webp",  # tag care se închide singur
# ], "Parserul nu a găsit exact imaginile așteptate"
# print("Testul a trecut!")
#
# response = requests.get(BASE_URL, timeout=TIMEOUT)
# finder = ImageFinder()
# finder.feed(response.text)
# print(len(finder.images), "imagini găsite")
# for src in finder.images:
#     print(src)
#
# print("Apariții de <img în HTML:", response.text.lower().count("<img"))

```

Fig 9 – Modules/2

```python

# --Ex40--

import hashlib

def page_fingerprint(url: str) -> str:
    response = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(response.content).hexdigest()


# first = page_fingerprint(BASE_URL)
# time.sleep(1)
# second = page_fingerprint(BASE_URL)
#
# print(first)
# print(second)
# print("Identice" if first == second else "Diferite")

# Amprenta depinde de continutul paginii , acelasi continut
# o sa rezulte on aceasi amprenta, dar orice modificare ar schimbao
# cybercor.org este o pagina statica astfel amprentele sunt indentice.
# Ca sa obtinem uo amprenta noua pagina trebuie sa fie modificata , ori
# trebuie noi sa modificam requestul spre pagina (alti parametrii de cerere)


# --Ex41--

response = requests.get(BASE_URL, timeout=TIMEOUT)

with open("headers.json", "w") as file:
    json.dump(dict(response.headers), file, indent=2)

with open("headers.json") as file:
    loaded = json.load(file)

# print(loaded.get("Content-Type", "lipsește"))

# Output: text/html; charset=utf-8


# --Ex42--

import socket

def resolve(hostname):
    return socket.gethostbyname(hostname)

# print(resolve("cybercor.org"))


# --Ex43--

def cert_days_left(hostname):
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as tls:
            cert = tls.getpeercert()
    expires = ssl.cert_time_to_seconds(cert["notAfter"])
    return int((expires - time.time()) // 86400)


# print(cert_days_left("cybercor.org"))


#----------------------------
```

Fig 10 – Modules/3

5.Part 5 – Main.py

```python
import webtools
import argparse
import csv
from datetime import datetime
from webtools import get_title, security_headers, DEFAULT_HEADERS
from webtools import fetch
from webtools import check_paths
from bs4 import BeautifulSoup

BASE_URL = "https://cybercor.org"

# --Ex44--

# print(webtools.get_title(webtools.fetch(BASE_URL).text))


# --Ex45--

# Daca main.py ar avea o functie proprie get_title, functiile sar
# ciocni, python tine un singur get_title in fisier, cel care este definit
# ultimul. Astfel daca def get_title apare dupa import el ar fi cel folosit
# si vice versa. Folosinf import webtools si webtools.get_title, problema nu apare


# --Ex46--

# Testerul ruleaza doar in webtools deoarece if-ul veririfica daca
# programul in care este rulat este programul principal.
# Acesta nu ruleaza in main.py deoarece este un import si nu indeplineste conditia.


# --Ex47--

# print(DEFAULT_HEADERS)
# print(get_title(fetch(BASE_URL).text))


# --Ex48--

parser = argparse.ArgumentParser(description="Analizează o pagină web.")
parser.add_argument("url", help="adresa paginii, de exemplu https://cybercor.org")
args = parser.parse_args()

url = args.url

# print(get_title(fetch(url).text))
# print(security_headers(url))


# Output:

# danu1@Danus-MacBook-Air web_lab % python main.py https://cybercor.org
# CYBERCOR
# {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}

```

Fig 11 – Main/1

```python
# --Ex49--

# parser = argparse.ArgumentParser(description="Verifică căi de pe un site.")
# parser.add_argument("url", help="adresa site-ului, de exemplu https://cybercor.org")
# args = parser.parse_args()
#
# statuses = check_paths(args.url, ["/", "/robots.txt", "/sitemap.xml"])
#
# with open("report.csv", "w", newline="") as file:
#     writer = csv.writer(file)
#     writer.writerow(["path", "status", "checked_at"])
#     for path, status in statuses.items():
#         writer.writerow([path, status, datetime.now().isoformat()])
#
# print("Raport salvat în report.csv")


# --Ex50--

from webtools import site_report

parser = argparse.ArgumentParser(description="Raport despre un site web.")
parser.add_argument("url")
args = parser.parse_args()

site_report(args.url)


# --BONUS--

def get_title(html: str) -> str:
    soup = BeautifulSoup(html, "html.parser")
    return soup.title.get_text(strip=True) if soup.title else ""

def extract_links(html: str) -> list:
    soup = BeautifulSoup(html, "html.parser")
    links = [tag["href"] for tag in soup.find_all(href=True)]
    return list(dict.fromkeys(links))



#--------------------------------
```

Fig 12 – Main/2

6. Part 5 – Webtools.py

```python
"""webtools: funcții reutilizabile pentru analiza paginilor web."""

import requests
import time
import json
import re
import socket
import ssl
import time
from typing import Optional
from urllib.parse import urlparse, urljoin
from bs4 import BeautifulSoup

TIMEOUT = 5

SECURITY_HEADERS = [
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "X-Frame-Options",
    "X-Content-Type-Options",
    "Referrer-Policy",
]

DEFAULT_HEADERS = {"User-Agent": "WebLab-danu"}

def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    """Întoarce obiectul Response pentru url, trimițând DEFAULT_HEADERS."""
    return requests.get(url, timeout=timeout, headers=DEFAULT_HEADERS)


def get_status(url: str) -> int:
    """Întoarce codul de stare HTTP al paginii."""
    return fetch(url).status_code


#def get_title(html: str) -> str:
    """Întoarce titlul paginii dintr-un șir HTML."""
    start = html.find("<title>") + len("<title>")
    end = html.find("</title>")
    return html[start:end].strip()

def get_title(html: str) -> str:
    soup = BeautifulSoup(html, "html.parser")
    return soup.title.get_text(strip=True) if soup.title else ""


def security_headers(url: str) -> dict:
    """Întoarce {antet: True/False} pentru cele cinci antete de securitate."""
    response = fetch(url)
    return {name: name in response.headers for name in SECURITY_HEADERS}

#if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
    print(__name__)

def check_paths(base: str, paths: list) -> dict:
    """Întoarce un dicționar {cale: cod_de_stare} pentru fiecare cale dată."""
    results = {}
    for path in paths:
        results[path] = get_status(base + path)
        time.sleep(1)
    return results

#def extract_links(html: str) -> list:
    """Întoarce legăturile (href) din HTML, fără duplicate."""
    links = re.findall(r'href="([^"]+)"', html)
    return list(dict.fromkeys(links))

def extract_links(html: str) -> list:
    soup = BeautifulSoup(html, "html.parser")
    links = [tag["href"] for tag in soup.find_all(href=True)]
    return list(dict.fromkeys(links))

```

Fig 13 – webtools/1

```python

def split_links(links: list, base: str) -> tuple:
    """Întoarce (interne, externe) față de domeniul lui base."""
    domain = urlparse(base).netloc
    internal, external = [], []
    for link in links:
        full = urljoin(base, link)
        if urlparse(full).netloc == domain:
            internal.append(full)
        else:
            external.append(full)
    return internal, external


def fetch_robots(base: str) -> Optional[str]:
    """Întoarce textul /robots.txt sau None dacă lipsește."""
    try:
        response = fetch(base + "/robots.txt")
    except requests.RequestException:
        return None
    if response.status_code != 200:
        return None
    return response.text


def disallowed_paths(robots_text: Optional[str]) -> list:
    """Întoarce valorile Disallow: din robots.txt."""
    if robots_text is None:
        return []
    paths = []
    for line in robots_text.splitlines():
        if line.strip().lower().startswith("disallow:"):
            value = line.split(":", 1)[1].strip()
            if value:
                paths.append(value)
    return paths


def score_headers(results: dict) -> str:
    """Întoarce scorul antetelor, de forma "3/5"."""
    return f"{sum(results.values())}/{len(results)}"


def resolve(hostname: str) -> str:
    """Întoarce adresa IPv4 a unui domeniu."""
    return socket.gethostbyname(hostname)


def cert_days_left(hostname: str) -> int:
    """Întoarce zilele rămase până la expirarea certificatului TLS."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as tls:
            cert = tls.getpeercert()
    expires = ssl.cert_time_to_seconds(cert["notAfter"])
    return int((expires - time.time()) // 86400)

```

Fig 14 – webtools/2

```python

def site_report(url: str) -> dict:
    """Colectează informații despre site, le afișează și le salvează în report.json."""
    hostname = urlparse(url).netloc
    response = fetch(url)
    html = response.text

    # Lanțul de redirecționări pornește de la http://, nu de la url-ul dat
    http_response = fetch("http://" + hostname)
    redirects = [f"{r.status_code} -> {r.headers.get('Location', '?')}"
                 for r in http_response.history]

    internal, external = split_links(extract_links(html), url)

    report = {
        "url": url,
        "status": response.status_code,
        "final_url": response.url,
        "title": get_title(html),
        "ip": resolve(hostname),
        "redirects": redirects,
        "security_score": score_headers(security_headers(url)),
        "cert_days_left": cert_days_left(hostname),
        "internal_links": len(internal),
        "external_links": len(external),
        "disallowed": disallowed_paths(fetch_robots(url)),
    }

    print(f"=== Raport site: {url} ===")
    print(f"Cod de stare:      {report['status']}")
    print(f"URL final:         {report['final_url']}")
    print(f"Titlu:             {report['title']}")
    print(f"Adresă IP:         {report['ip']}")
    print(f"Redirecționări:    {', '.join(redirects) if redirects else 'niciuna'}")
    print(f"Scor securitate:   {report['security_score']}")
    print(f"Certificat:        {report['cert_days_left']} de zile rămase")
    print(f"Legături:          {report['internal_links']} interne, {report['external_links']} externe")
    print(f"Căi interzise:     {', '.join(report['disallowed']) if report['disallowed'] else 'niciuna'}")

    with open("report.json", "w") as file:
        json.dump(report, file, indent=2, ensure_ascii=False)
    print("Salvat în report.json")
    return report
```

Fig 15 – webtools/3

Concluzie

În acest laborator am învățat să descarc și să analizez pagini web cu requests, să extrag informații din HTML și să organizez codul în funcții cu o singură sarcină, cu docstringuri și adnotări de tip.

Am construit apoi un modul propriu, webtools.py , importat din main.py , cu argumente din linia de comandă arg parse și salvarea rezultatelor în CSV și JSON. Proiectul final, combină funcțiile într-un raport despre un site: cod de stare, titlu, IP, redirecționări, antete de securitate, certificat, legături.